

제 10회 2026 미래에셋증권 AI Festival

Hybrid-DB-Query

금융상품 AGENT부문
TEAM 금융야호 | 김민섭 김이정 윤왕규

목차

- 01

전체 아키텍처 및 특화 기능

시스템 전체 아키텍처와 특화 기능
- 02

주요 기능 흐름도

적재 > 색인 > 검색 > 생성 파이프라인
- 03

제안 방법

데이터 처리 / 온톨로지 엔지니어링 / 검색 방법론
- 04

기대 효과 및 확장성

PB 금융상품 검색 채팅 서비스로 확장
- 05

부록

참고 논문 및 활용 파트

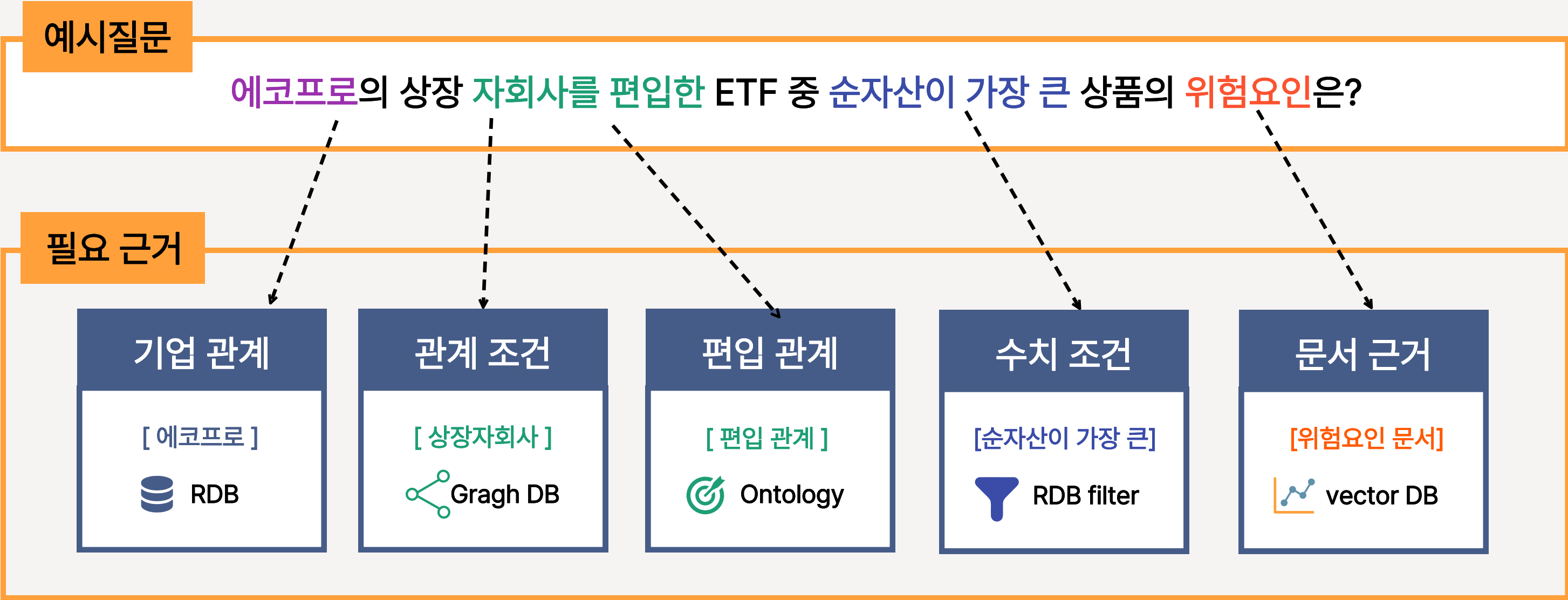
01

전체 아키텍처 및 특화기능

| 문제정의

숨겨진 금융 컨텍스트를 시스템이 처리할 수 있는 하드 데이터 변수로 해체하여 Hybrid-DB-Query를 실행한다

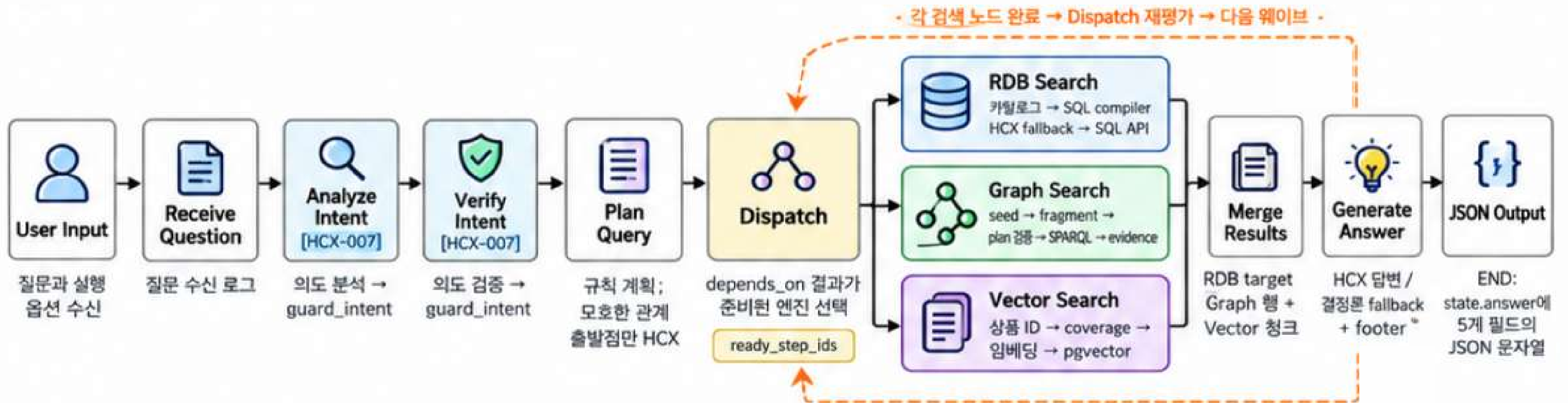
수치 조건·기업 관계·투자설명서 근거를 함께 조회하는 금융 온톨로지 기반 질의응답 Agent



| 프로젝트 아키텍처

Query 처리 Flow

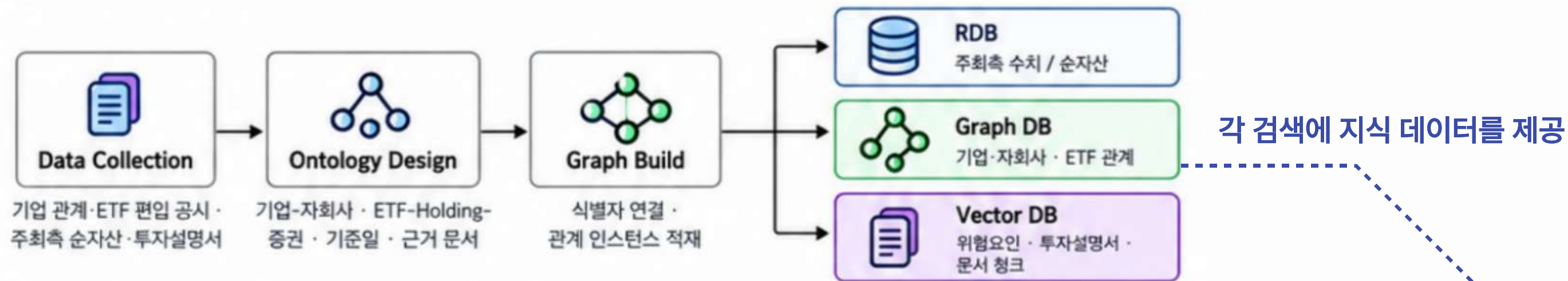
사용자 질문을 분석하고, 최적의 데이터를 스스로 검색하여 근거 기반 답변을 생성하는 실시간 파이프라인



| 프로젝트 아키텍처

외부 데이터 적재 Flow

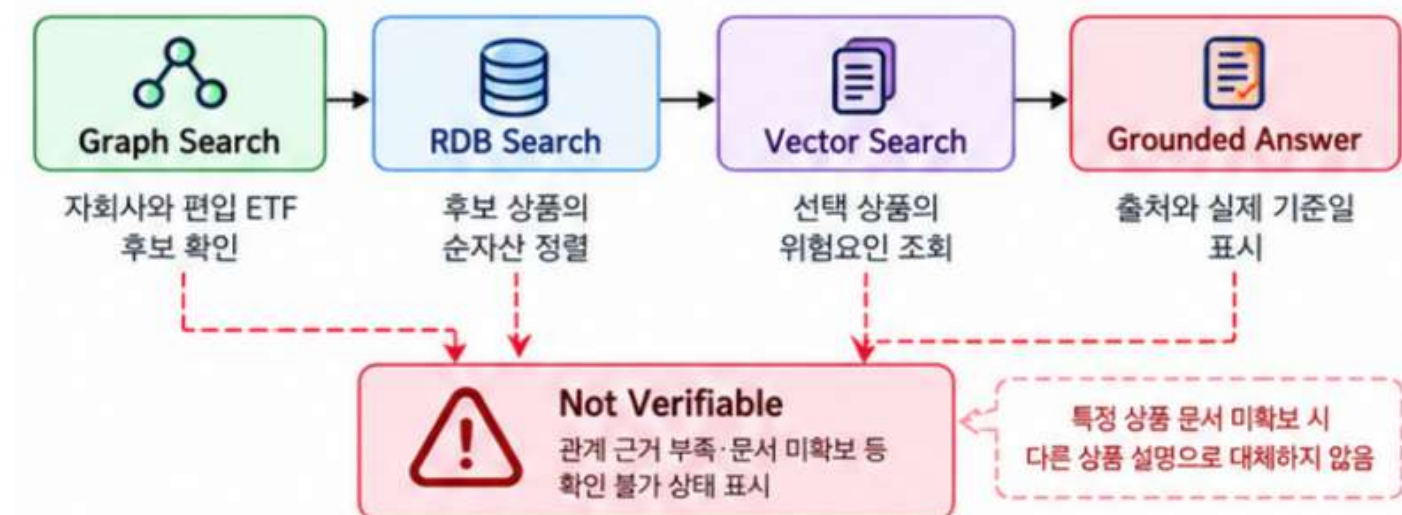
외부데이터를 수집/정제하고, 온톨로지 기반으로 연결하여 Gragh/RDB/Vector 검색에 활용하는 지식 구축 파이프라인



[예시 질문 처리 Case]

에코프로의 상장 자회사를 편입한 ETF 중 순자산이 가장 큰 상품의 위험요인은?

- 1 Gragh가 자회사 관계와 편입 ETF 후보를 찾음
- 2 RDB가 주최측 순자산 컬럼으로 정렬
- 1 Vector는 선택 상품 문서에서 위험 요인을 검색



| 프로젝트 아키텍처

특화 기능

기술적 차별화는 LLM이 모든 검색 방법과 계산을 자유롭게 결정하도록 두지 않는 데 있다
금융상품 조건의 결정적 SQL 변환, 기업·편입 관계 검증, 상품별 문서 검색, 기준일·출처 관리, 확인 불가 응답으로 관리한다

비교 대상	주요 한계	본 시스템의 대응
순수 LLM 응답	수치·상품 존재 여부·관계에 대한 조회시 근거 부족	실제 저장소에서 조건과 관계를 확인한 결과로 답변 출력
일반 문서 RAG	문서 검색만으로는 수치 정렬과 관계 방향을 검증하기 어려움	결정적 RDB를 검색 + 온톨로지를 검증하는 Gragh + 상품 범위 제한 Vector검색을 결합함
자유형 Text2SQL·Planner	엔진 선택/의존성/물리 스키마 생성 오류	검증된 속성 카탈로그와 규칙기반 실행계획을 기본 경로로 사용함

02

주요 기능 흐름도

| Query 처리 Flow

사용자 질문을 분석하고, 최적의 데이터를 스스로 검색하여 근거 기반 답변을 생성하는 실시간 파이프라인

1. User input

- 사용자가 입력한 질문을 받고, 어떤 질문이 들어왔는지 기록한다
- question과 선택 항목인 question_id, max_sql_retries를 받아 질문 수신 로그를 남긴다

2. Intent Analysis

- LLM(HCX-007+Output Structure)이 질문에서 조회 대상 상품군과 상품·기업명, 필터 조건, 개체 간 관계, 정렬 기준·조회 개수, 답변에 필요한 항목을 추출해 구조화한다
- 실제 DB·컬럼 선택과 조회는 다음 단계에서 수행한다
- Json format의 {Query_frame}을 추출한다

3. Query Planning

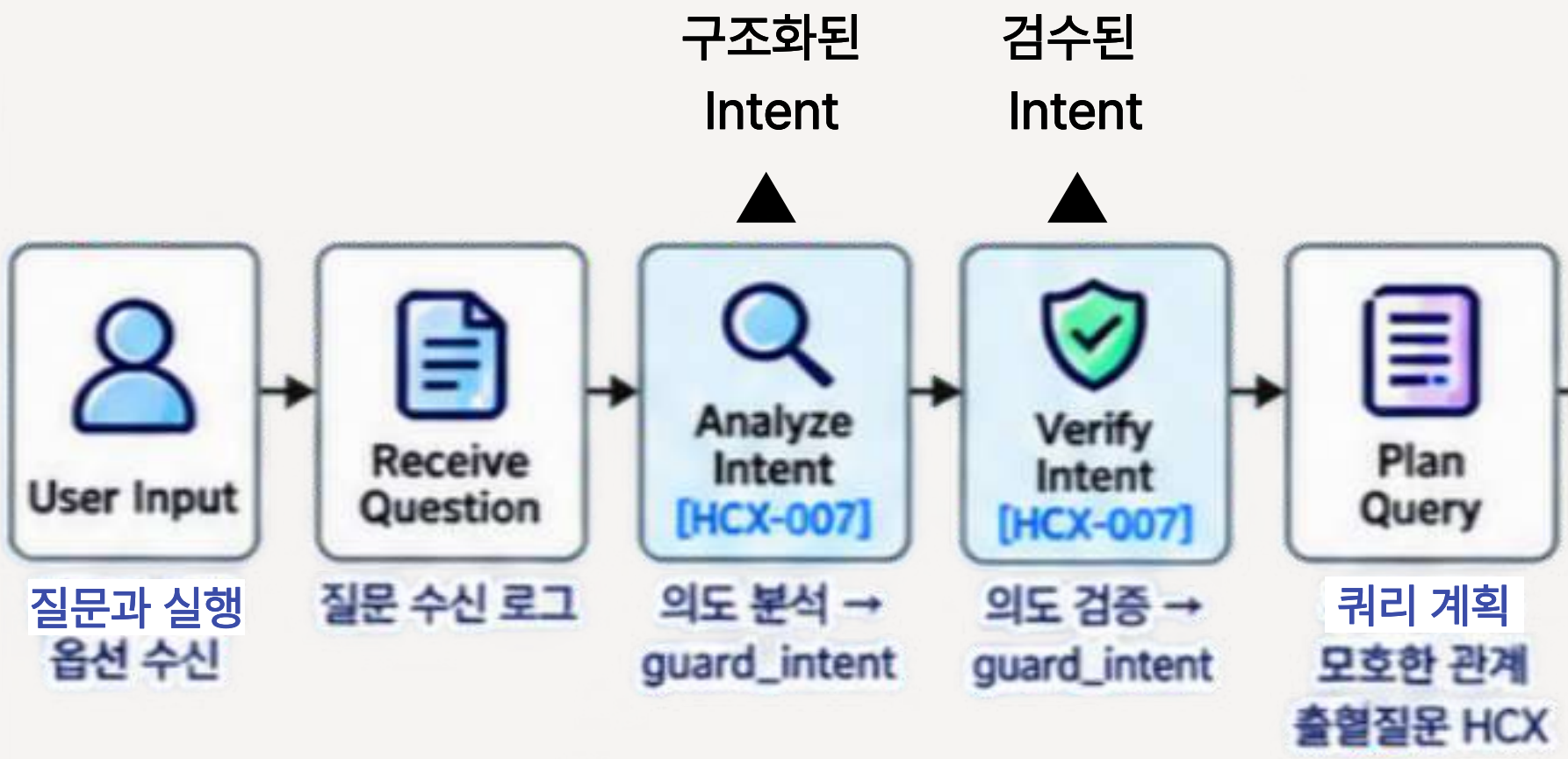
- Intent의 관계·필터 조건·정렬 기준·설명 요청을 분석해 Graph·RDB·Vector별 조회 작업을 구성한다.
- 각 작업이 필요로 하는 선행 결과를 지정하여 검색 순서와 독립 실행 가능한 작업을 결정한다

4. Intent Verification

- HCX-007이 원문과 1차 Intent를 비교해 누락된 조건·관계와 잘못된 분류를 교정한다
- 이후 규칙 기반 Guard가 연산자·관계·테마·중복 항목을 정규화해 실행 가능한 Intent로 확정한다

5. Query Planning

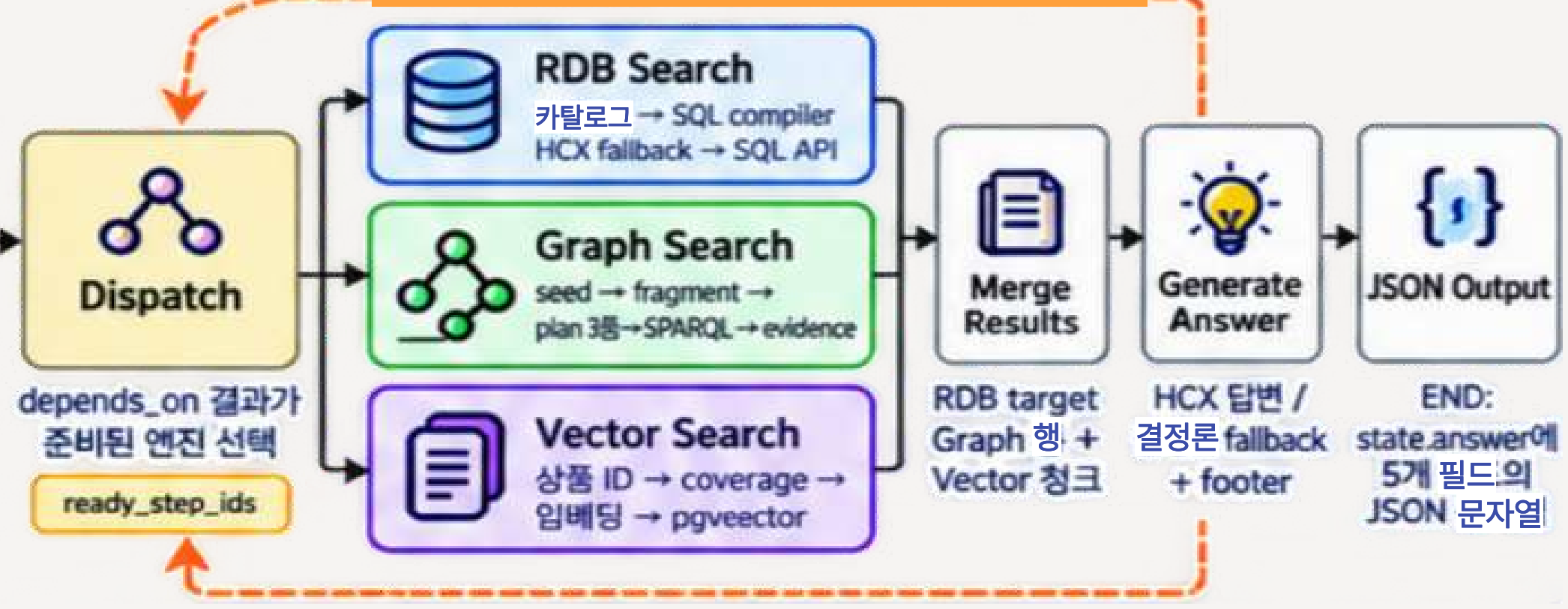
- Intent를 바탕으로 검색 엔진과 단계별 의존관계를 결정한다



Query 처리 Flow

사용자 질문을 분석하고, 최적의 데이터를 스스로 검색하여 근거 기반 답변을 생성하는 실시간 파이프라인

※ 검색 노드가 완료될 때마다 준비 상태를 재평가



9. Answer Generation

- HCX-007이 검색된 값과 문서로 답변을 작성한다

8. Result Merge

- RDB 최종 조회 행, Graph 관계 행, Vector 문서 청크를 출처 태그와 함께 모은다

7-1. Graph Search

- 이름·코드를 개체로 해소하고, 온톨로지 스키마와 관계 계획을 검증해 SPARQL을 실행한다.

7-2. RDB Search

- 카탈로그로 개념을 컬럼에 연결하고 SQL을 컴파일한다

7-3. Vector Search

- 상품 ID와 문서 확보 상태를 확인하고, 요청 주제를 임베딩해 해당 상품·섹션의 문서를 검색한다

6. Wave Dispatch

- 이름·코드를 개체로 해소하고, 온톨로지 스키마와 관계 계획을 검증해 SPARQL을 실행한다.

| 질문 분석 개념 단위 분해

목적 : SQL과 자연어 사이의 간극을 메우기 위해 DB 구조를 모른 채 질의를 개념 단위로 먼저 분해해 중간표현을 설계하는 것
설계 : 실제 컬럼명 대신 개념명만 다루도록

참고 문헌

1. Guo et al., "Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation (IRNet)", ACL 2019 : SQL과 자연어 사이의 간극을 메우기 위해 SemQL이라는 중간표현을 설계했다는 부분에서 참고

2. Wolfson et al., "Break It Down: A Question Understanding Benchmark", TACL 2020 : 질문을 특정 질의 언어와 무관하게 개념 단위 자연어 단계(QDMR)로 표현한다는 부분에서 참고

3. Pourreza & Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction", NeurIPS 2023 : 질의 생성을 여러 하위 단계로 나누고 각 단계 해법을 다음 단계에 넘기는 방식이 정확도를 유의미하게 높인다는 부분에서 참고

주요 키	역할
task	질문 성격 분류
product_domain	상품군과 하위유형을 한 객체로 묶어 모호성 제거
target_entities	질문이 직접 지목한 상품명·회사명
conditions	단순 속성 필터
relations	엔티티 간 관계
sort	정렬·TOP N
output_requirements	결과 필수 필드 + 서술형 설명이 필요한 주제 표시

I 분석 검증 LLM-as-a-Judge

목적 : Query intent 추출 결과를 별도 LLM 호출로 검토

설계 : 조건 누락, relations 누락, 최상급 표현 오분류 등 구체적 실패 유형별로 교정

예시 (질문 분석 결과)

`{"raw_question": "장내에서 거래되는 은행채 중에서, 듀레이션이 2년 이하인 상품을 평가일단가가 낮은 순서대로 3개만 보여줘.",`

`"product_domain": [{"domain": "채권", "subtype": ["은행채"]}],
"conditions": [{"attribute": "듀레이션",
 "operator": "lte",
 "value": "2년",
 "value_2": "",
 "applies_to": "target",
 "domain": "채권"}],`

`"sort": {"attribute": "평가일단가",
 "order": "asc",
 "limit": "3"},`

`"output_requirements": "fields": ["평가일단가"]}`



LLM 검증 결과 누락된 조건 파악 후 자동 추가

`{"raw_question": "장내에서 거래되는 은행채 중에서, 듀레이션이 2년 이하인 상품을 평가일단가가 낮은 순서대로 3개만 보여줘.",`

`"product_domain": [{"domain": "채권", "subtype": ["은행채"]}],
"conditions": [{"attribute": "듀레이션",
 "operator": "lte",
 "value": "2년",
 "value_2": "",
 "applies_to": "target",
 "domain": "채권"},
 {"attribute": "시장구분",
 "operator": "eq",
 "value": "장내",
 "value_2": "",
 "applies_to": "target",
 "domain": "채권"}],`

`"sort": {"attribute": "평가일단가",
 "order": "asc",
 "limit": "3"},`

`"output_requirements": "fields": ["평가일단가"]}`

| DB 검색 흐름 결정 Wave Dispatch

※ 37p DB 라우팅 전략에서 자세히 설명합니다

목적 : 질문 분석 노드가 만든 intent만 보고, RDB/GraphDB/VectorDB 중 무엇을 어떤 순서로 조회할지 계획

설계 : 규칙 기반이 기본, LLM은 예외 1건만

핵심

- intent 구조(relations의 object_ref 체인, conditions.applies_to/domain, product_domain, sort)만 보면 대부분 규칙으로 결정됨
- 예외는 relation의 주체가 구체적 이름도, 다른 relation의 결과(object_ref)도 아닌 경우
- 이때는 LLM을 호출해 "RDB를 먼저 조회해야 하는지"를 판단
- LLM 호출이 실패하거나 쓸 수 없으면 "RDB 선행 조회 불필요"라는 안전한 기본값으로 처리
- 독립된 엔진은 병렬 실행하고, 의존 단계는 다음 웨이브로 넘김

예시

Q : "신용등급 A+ 이상인 채권을 발행한 회사의 자회사를 편입한 ETF"

이 질문은 "자회사" relation의 주체(모회사)가 이름도 object_ref도 아닌, RDB 조건을 만족하는 채권 발행사 목록(LLM fallback이 걸리는 케이스)

1. rdb_pre_R1 (entity_lookup) : RDB에서 신용등급 A+ 이상 채권의 발행사 목록 조회. LLM이 needs_rdb_prestep=true로 판단해 생성
2. graph_R1 (자회사) : depends_on=[rdb_pre_R1]. 위 발행사들의 자회사를 GraphDB에서 조회
3. graph_R2 (편입) : depends_on=[graph_R1]. 그 자회사를 편입한 상품을 조회
4. rdb_국내ETF, rdb_해외ETF (target) : depends_on=[graph_R2]. 최종 ETF 목록과 필드 조회

route 요약: needs_graph=true, needs_rdb=true, used_llm_fallback=true, llm_fallback_details=[{relation_id: R1, rdb_domain: 채권, reasoning: "..."}]

| RDB 검색 과정 NL2SQL

1. 테이블 매핑



2. 컬럼 선택



3. SQL 초안 생성



4. SQL문 생성



5. SQL 실행

Q : 장내에서 거래되는 은행채 중에서, 듀레이션이 2년 이하인 상품을 평가일단가가 낮은 순서대로 3개만 보여줘

1. 테이블 매핑: 채권 → 채권 table

2. 컬럼 선택

- 하위유형 "은행채":

SUBTYPE_CONDITION_MAP에서 직접 **std_pd_scls_nm** = '은행채'로 결정

- 듀레이션 → 카탈로그 키와 정확히 일치해 바로 **dur**로 매칭

- 시장구분 → 실제 컬럼 목록의 **pd_exg_mkt** 설명("거래구분 값은 장내/장외")과 대조되어 **pd_exg_mkt**로 매칭

- 평가일단가 → **eval_price** 설명("평가일단가(Clean Price 성격)")과 정확히 일치해 **eval_price**로 매칭

- 상품명(**pd_no, pd_nm**) → target 조회라 강제 포함. 채권은 종목 하나가 여러 행으로 나올 수 있다는 캐비어트 때문에 **DISTINCT pd_no**도 함께 적용

3. SQL 자연어 초안 절차를 다음과 같이 설명:

- 1. **상품 유형 필터링**: **'std_pd_scls_nm'** 컬럼을 사용하여 상품 유형이 '은행채'인 행들만 선택합니다.
- 2. **시장 구분 필터링**: **'pd_exg_mkt'** 컬럼을 사용하여 시장 구분이 '장내'인 행들만 선택합니다.
- 3. **듀레이션 필터링**: **'dur'** 컬럼 값을 확인하여 듀레이션이 2년 이하인 행들을 선택합니다. 여기서 결측치는 고려하지 않습니다.
- 4. **평가일단가 필터링 및 정렬**: **'eval_price'** 컬럼이 NULL이 아닌 행들만 선택합니다. 그런 다음, **'eval_price'** 컬럼을 기준으로 오름차순 정렬합니다.
- 5. **상위 3개 선택**: 정렬된 결과에서 상위 3개의 행만을 선택합니다.
- 6. **출력 필드 지정**: 최종 결과는 각 상품의 **'pd_nm'** (상품명)와 해당 상품의 **'eval_price'** (평가일단가) 정보를 포함하여 출력됩니다.

4. SQL 생성

```
SELECT DISTINCT pd_no, pd_nm, eval_price
FROM raw.prbd01n001
WHERE std_pd_scls_nm = '은행채'
AND pd_exg_mkt = '장내'
AND dur <= 2
AND eval_price IS NOT NULL
ORDER BY eval_price ASC
LIMIT 3;
```

5. SQL 실행 및 실패 시 N회 재시도

[원본 질문], [해석된 스키마], [실제 컬럼 전체 목록], [실패한 SQL], [DB 에러]를 넣고 재작성 → 재실행 반복

GraghDB 검색 과정 NL2SPARQL

1. 의미 구조화



2. 엔티티 확정



3. 관계 경로 설계



4. 검증·질의 컴파일



5. 근거 확정·전달

Q : 엔비디아를 편입한 국내 ETF와 해외 ETF 중 순자산이 큰 상품을 비교해줘

1. intent 의미 구조화 : 질문을 검색 가능한 구조로 분해

- 대상 상품: 국내 ETF + 해외 ETF
- 관계 조건: ETF --편입(holds)--> 엔비디아
- 정렬: 순자산 내림차순
- > 아직 어떤 테이블이나 Graph property를 사용할지 결정하지 않고, 질문의 의미만 정리

2. 엔티티 확정

- 엔비디아를 GraphDB의 기업 또는 증권 엔티티로 해소
- `resolve_frame_seed()`가 검색할 엔티티 역할을 정하고, `resolve_entity()`가 URI 후보를 찾음
- 후보가 여러 개면 임의로 선택하지 않고 ambiguous 상태로 중단(Graph 검색을 진행X)

엔비디아

→ fp:Company 또는 fp:Security

→ GraphDB의 고유 URI

3. 관계 경로 설계

- 엔비디아와 ETF가 어떤 경로로 연결되는지 계획
- 온톨로지의 property 방향에 맞춰 방향 전환

[핵심관계]

엔비디아 기업 ← issuedByCompany

증권 ← holdingSecurity

Holding ← hasHolding | ETF

[의존관계]

- 실행 계획은 국내 ETF와 해외 ETF에 공통으로 적용
- graph_R1: 엔비디아를 발행한 증권을 편입한 ETF 찾기
- rdb_국내ETF: 국내 ETF의 순자산 조회
- rdb_해외ETF: 해외 ETF의 순자산 조회

graph_R1

└─→ rdb_국내ETF

└─→ rdb_해외ETF

4. 검증/질의 컴파일

- 관계 계획을 실제 SPARQL과 SQL로 바꾸기 전에 검증
- : Graph 검증 RDB 검증

- Company, Security, Holding, ETF class가 schema fragment에 포함되는지 확인
- hasHolding, holdingSecurity, issuedByCompany가 허용된 ObjectProperty인지 확인
- 각 edge의 subject와 object가 property의 domain/range에 맞는지 확인
- seed인 엔비디아가 그래프 경로에 연결되어 있는지 확인
- 출력 항목에 ETF 코드와 상품명이 포함되어 있는지 확인

5. 근거 확정 및 전달

- 엔비디아 편입이 확인된 국내 ETF와 해외 ETF를 순자산 순으로 비교
- 상품별 국내·해외 구분과 수치 기준일 표시
- 근거가 부족한 항목은 별도로 "확인할 수 없음" 표시

| VectorDB 검색 과정 NL2SPARQL



Q : 미래에셋 글로벌성장 펀드의 운용 전략과 투자 위험을 설명해줘

DB : 해당 상품 Vector DB에 아래와 같은 문서가 존재

- **objective_strategy**
 - 예) 글로벌 성장주 중심으로 장기 자본 성장을 추구한다
- **risk**
 - 예) 주식 시장 변동과 환율 변동에 따라 원금 손실이 발생할 수 있다

1. 질문에서 검색 주제 분리

Intent와 Vector 단계는 질문을 "운용 전략"과 "투자 위험"의 두 주제로 분리

2. 상품명으로 검색 범위 확정

문의 상품명인 미래에셋 글로벌성장 펀드를 product_master에서 검색해 상품 ID로 바꿈

미래에셋 글로벌성장 펀드
→ fund:MGF

상품 ID가 확인됐기 때문에 전체 펀드 문서를 검색하지 않고 fund:MGF에 연결된 문서만 검색

3. 문서 확보 상태 확인

- fund:MGF에 연결된 문서가 있는지 확인
- 문서가 없었다면 Vector 검색을 진행하지 않고 no_document 상태가 됨

fund:MGF
→ matched_fund_doc
→ 문서 검색 가능

4. 검색 주제를 문서 섹션으로 변환

- 주제를 VectorDB의 섹션 유형으로 변환

운용 전략 → objective_strategy
투자 위험 → risk

- 그 결과, 검색 조건은 아래와 같음

product_id = fund:MGF
section_type IN ('objective_strategy', 'risk')

5. 주제별 임베딩 검색

- 주제마다 질문을 확장해 임베딩 검색을 수행

질의 1:

미래에셋 글로벌성장 펀드의 운용 전략과 투자 위험을 설명해줘 운용 전략

질의 2:

미래에셋 글로벌성장 펀드의 운용 전략과 투자 위험을 설명해줘 투자 위험

6. 유사도 처리와 중복 처리 후 최종 확정

- 유사도 0.45 이상인 청크만 답변 후보로 남김

strategy-1: 0.81 → 포함

risk-1: 0.74 → 포함

- 같은 청크가 두 주제 검색에서 중복으로 나오면 chunk_id 기준으로 하나만 남기고, 더 높은 점수를 사용

03

제안 방법

데이터 처리

어떤 특화 데이터를 어떻게 수집·정제했는가

I 사용 데이터



주최측 데이터
기존 보유한 상품·ETF·기본 정보



외부데이터 보강
편입관계 · 기업 연결 · 위험요인 · 이력

I 전체 구축 데이터 현재RDB + Schema Vector + Content Vector + TBox/ABox TTL 생성

계층	구축 대상	현재 구축 현황	주요 역할
PostgreSQL RDB	금융상품 Master·Enriched·Relations	raw·enriched·relations 11개 테이블 구축	수치 조회, 조건 필터, 집계·정렬, 상품·관계 데이터 관리
Schema Vector	DB Schema·Ontology 용어	bond_schema_terms 130건, schema_terms_all 188건	질의와 관련된 Table·Column·Ontology Schema 탐색
Content Vector	투자설명서·SEC 공시 등 비정형 문서	2,900문서 / 17,765청크 구축	상품 위험·운용전략·기초지수 등 Semantic Retrieval
Graph TBox	금융 도메인 Ontology	TBox TTL 5개 생성·검증 완료	상품·기업·관계 개념 및 Schema 정의
Graph ABox	상품·기업·관계 Instance	instances_*.ttl 5개 생성·검증 완료	RDB·관계 데이터를 RDF Instance로 표현

I 파생 / 관계 데이터

데이터	규모	주요 역할
bond_kr_enriched.csv	21,882 × 12	신용등급 Rank, 잔존일수, 통화·구매가능 여부 보강
etf_kr_enriched.csv	1,780 × 12	LSEG 기반 총보수·복제방식 보완
company_master.csv	118,709 × 5	DART·KIND 기반 상장·비상장 기업 식별
holding_code_map.csv	1,393 × 8	ETF 편입종목 코드 → 기업·ETF 연결
etf_holding.csv	47,016 × 7	ETF → 편입종목 관계
etf_theme.csv	5,646 × 4	ETF → 테마 관계
company_subsidiary.csv	30,097 × 10	기업 → 출자기업·자회사 관계

주요 관계 경로

관계 경로	활용
ETF → Holding → Company	ETF가 어떤 기업에 얼마나 노출되어 있는지 분석
Company → Subsidiary → Company	기업의 자회사·출자관계 분석
Bond → Issuer → Company	채권 발행사를 기업 식별체계와 연결
ETF → Theme	상품별 테마·산업 노출 분석

I 사용 데이터



주최측 데이터
기존 보유한 상품·ETF·기본 정보

+



외부데이터 보강
편입관계 · 기업 연결 · 위험요인 · 이력

I Content Vector DB 적재 현황

source_type	발행처	문서 수	청크 수	문서 기간
sec_edgar_dera_rr	미국 SEC EDGAR DERA	1,591	7,073	2025-07-01 ~ 2026-06-30
dart_prospectus_cover	금융감독원 DART	746	839	2015-03-18 ~ 2026-08-19
dart_etn_prospectus	금융감독원 DART	291	847	2023-10-10 ~ 2026-08-24
dart_prospectus	금융감독원 DART	197	1,837	2015-06-11 ~ 2026-08-07
dart_bond_prospectus	금융감독원 DART	44	790	2026-07-27 ~ 2026-08-24
manager_prospectus_pdf	자산운용사 직접 공시	31	6,379	2025-02-27 ~ 2026-07-30
합계		2,900	17,765	2015-03-18 ~ 2026-08-24

I 상품 도메인별 Content Vector 커버리지

도메인	주요 문서 유형	연결 문서
국내 ETF (etf_kr)	DART 투자설명서(집합투자증권)	744
국내 ETF (etf_kr)	운용사 투자설명서 PDF	31
국내 ETN (etn_kr)	일괄신고추가서류(파생결합증권)	291
공모펀드 (fund)	투자설명서(집합투자증권)	199
국내채권 (bond_kr)	투자설명서(일괄신고)·증권신고서	40
해외 ETF (etf_gl)	SEC 485BPOS·497·485APOS	1,591

온톨로지 엔지니어링

어떤 온톨로지 엔지니어링을 적용했는가

TBox 설계, 형식논리로 오답을 원천 차단하는 스키마

issuedBy domain 제한

- 문제: ETF를 주어로 발행 주체를 묻는 질의가 들어올 수 있음
- 해결: issuedBy의 domain을 Bond 단독으로 제한하여 ETF를 주어로 놓으면 데이터 조회 없이 형식논리만으로 모순이 확정되어 즉시 답변 보류로 처리

```
fp:issuedBy a owl:ObjectProperty ;
rdfs:domain fp:Bond ;
rdfs:range fp:Issuer .
```

ETN 클래스 분리

- 문제: 국내 ETF 원천에 ETN 545종이 섞여 있으나 ETN은 편입종목과 NAV 개념이 없음
- 해결: ETN을 ETF와 별도 클래스로 분리하여 ETF 개수 집계 가 약 44% 부풀려짐 방지

fp:ETF ↔ fp:ETN owl:disjointWith 선언

sameVehicleAs

- 문제: 공모펀드와 ETF는 서로소 클래스이나 동일 종목이 양쪽에 중복 등록되어 있음
- 해결: owl:sameAs 대신 sameVehicleAs 대칭 관계로 연결하여 서로소 제약을 유지한 채 동일 실체를 표현함

```
fp:sameVehicleAs a owl:ObjectProperty,
owl:SymmetricProperty ;
rdfs:domain fp:Product ;
rdfs:range fp:Product .
```


TBox 설계, 형식논리로 오답을 원천 차단하는 스키마

전이성 미션언

- 문제: 지분율이 출자관계마다 달라 자회사 관계가 자동으로 전이되지 않음
- 해결: fp:hasSubsidiary에 owl:TransitiveProperty를 붙이지 않고, 손자회사 탐색은 SubsidiaryRelation을 거치는 다단계 SPARQL 질의로 처리

지분율은 곱으로 줄어들어($60\% \times 60\% = 36\%$) 관계가 그대로 이어지지 않으므로, **hasSubsidiary**는 비전이적으로 두고 손자회사는 다단계 SPARQL로 **fp:ownershipPct**를 직접 확인

신용등급 개체화

- 문제: 문자열 사전순 비교로는 BB+가 AA-보다 크게 나와 등급 순서가 뒤집힘
- 해결: 신용등급 19단계를 개체화하고 ratingRank 정수를 부여

문자열 사전순으로 비교하면 'BB+' > 'AA-'가 되어 등급 순서가 뒤집히기 때문에 "AA- 이상" 처리는 **ratingRank <= 4**로 판정

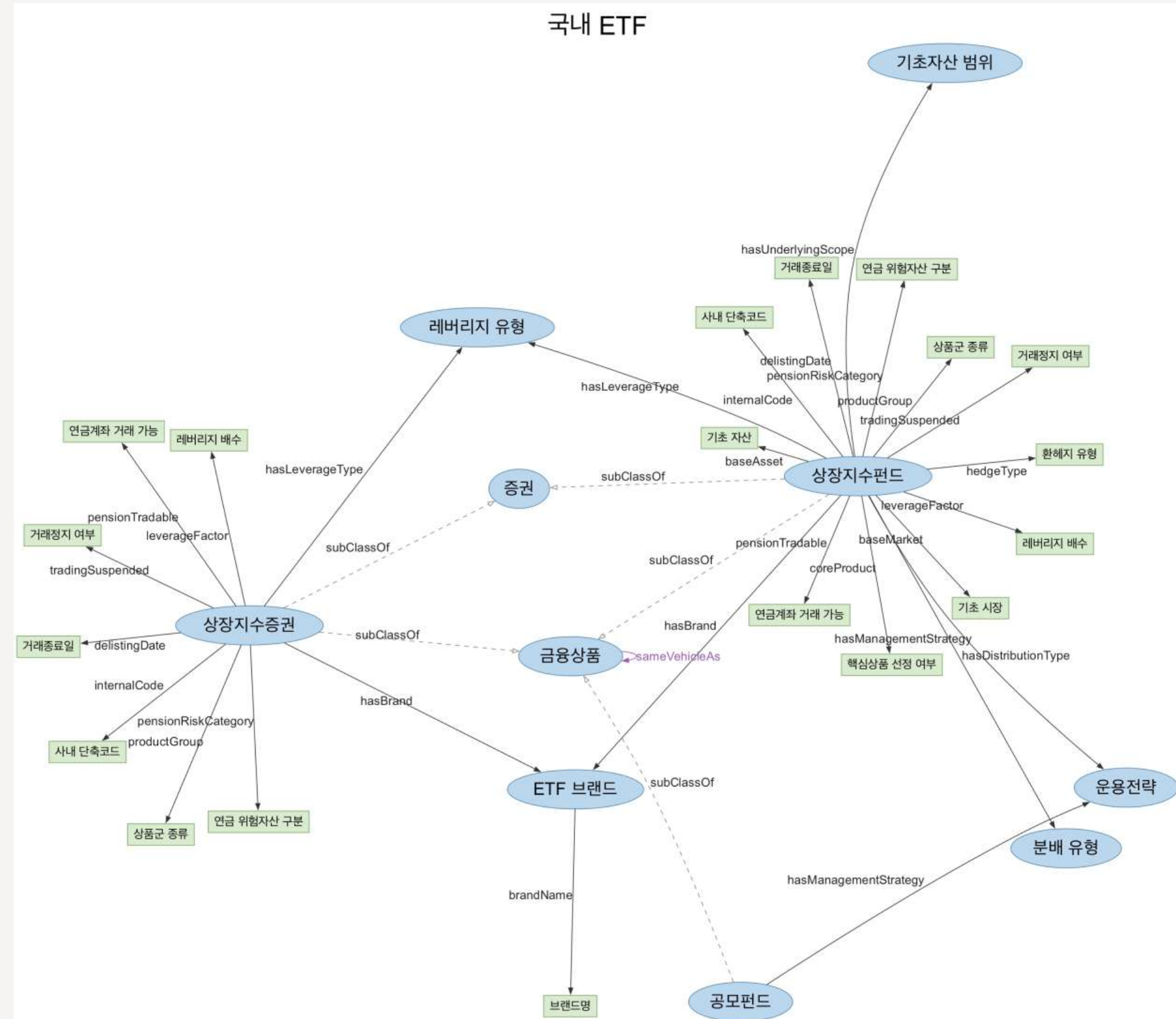
위험등급 통합

- 문제: 위험등급이 채권, 국내ETF, 공모펀드에서 컬럼명과 코드 체계가 전부 다름
- 해결: 서열이 1을 최고위험으로 두는 점이 동일함을 확인해 6개 통합 개체로 묶음

fp:RiskGrade_2 a **fp:RiskGrade** ;
rdfs:label "높은 위험(2등급)"@ko ;
skos:altLabel "높은위험(2등급)"@ko,
"PD_RISK_GCD_12"@en, "2"@en ;
fp:riskGradeLevel 2 .

예시 1) 국내 ETF

1. 레버리지 유형·ETF 브랜드 화살표가 ETF·ETN 양쪽에서 나오는 것은 **hasLeverageType·hasBrand**의 domain이 union(ETF, ETN)이기 때문
2. 공모펀드가 등장하는 것은 **hasManagementStrategy**의 domain이 union(ETF, PublicFund)이기 때문
3. 금융상품(Product) 위 보라색 self-loop는 **sameVehicleAs**가 유일한 다리 관계라서 생김



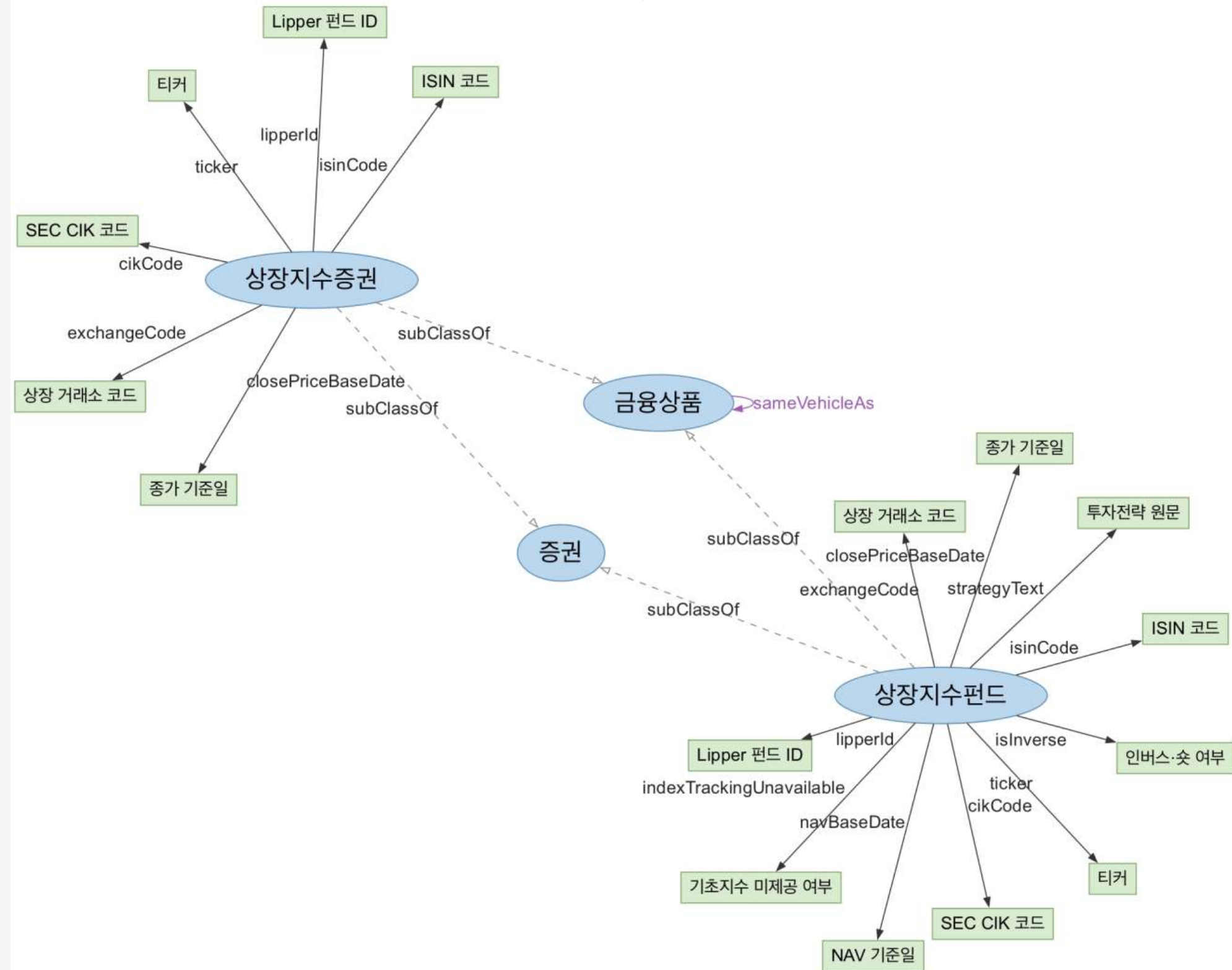
| TBox 설계, 형식논리로 오답을 원천 차단하는 스키마

예시 2) 해외 ETF

- 파란 타원 → 클래스(owl:Class)
- 화살표 → 관계(object property)
 - 검정 = 파일 로컬 선언
 - 보라 = common.ttl에서 온 다리 관계
- 클래스에 딸린 작은 초록 박스 → 속성(datatype property)
- 회색 점선 → 상속(rdfs:subClassOf)

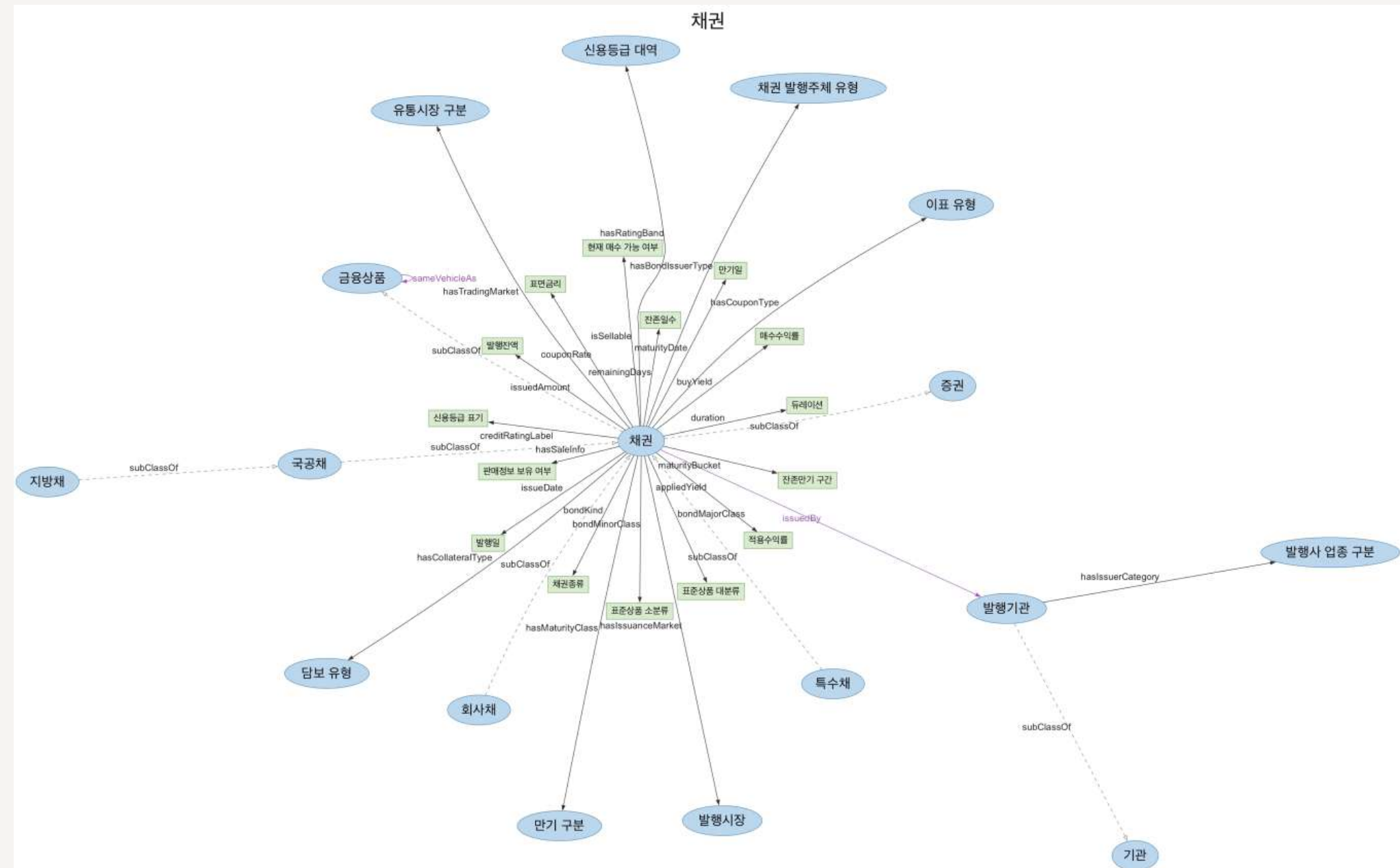
1. ETF 전용 속성 4개(strategyText, isInverse, indexTrackingUnavailable, navBaseDate)는 ETN 화살표가 없음. domain이 fp:ETF 단독이기 때문
2. ETN·ETF 모두 증권·금융상품으로 향하는 subClassOf 점선이 있어, 별도 화살표 없이도 두 클래스가 분리되어 있음이 확인됨

해외 ETF



예시 3) 채권

1. subClassOf가 2단으로 이어짐: 국공채 → 채권, 지방채 → 국공채. 지방채는 채권에 직접 연결되지 않고 국공채를 거쳐서만 연결됨
2. 채권에서 한 번 더 뺄어나가는 2홉 구조가 있음: 채권 → 발행기관 → 기관(subClassOf), 발행기관 → 발행사 업종 구분(hasIssuerCategory). ETF·ETN 그림에는 없던 패턴

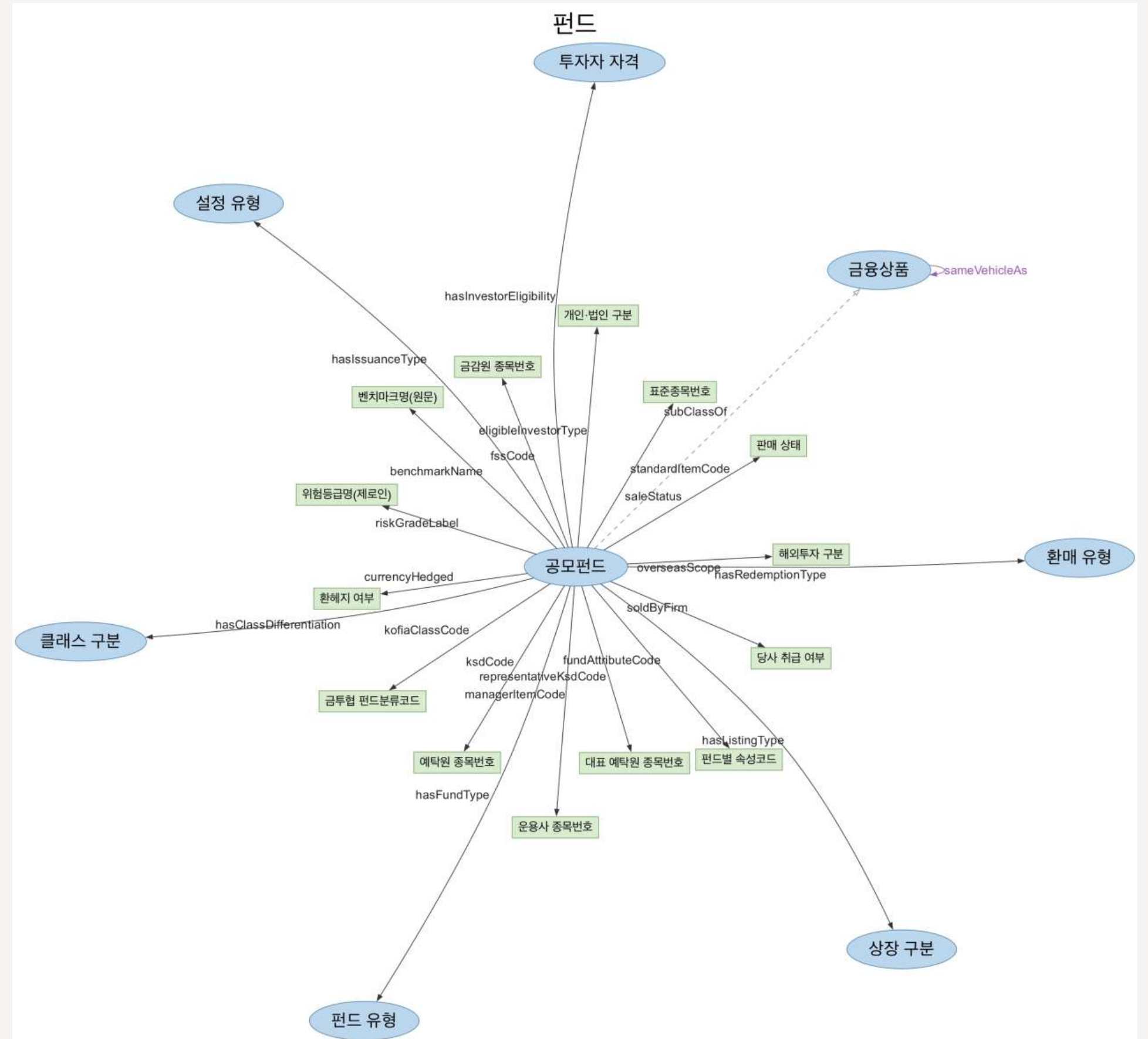


TBox 설계, 형식논리로 오답을 원천 차단하는 스키마

예시 4) 펀드

- 파란 타원 → 클래스(owl:Class)
- 화살표 → 관계(object property)
 - 검정 = 파일 로컬 선언
 - 보라 = common.ttl에서 온 다리 관계
- 클래스에 딸린 작은 초록 박스 → 속성(datatype property)
- 회색 점선 → 상속(rdfs:subClassOf)

- 공모펀드 → 금융상품 subClassOf 선언 있고, 증권(Security)으로 가는 선이 없음. ETF·ETN·채권은 전부 증권으로도 연결됐던 것과 달리 공모펀드는 **금융상품만 상속하고 증권은 상속하지 않는 구조**
- 초록 박스(datatype property)가 다른 그림보다 훨씬 많고, 그 중 예탁원 종목번호·대표 예탁원 종목번호·운용사 종목번호·금감원 종목번호·표준종목번호처럼 **서로 다른 식별코드가 나란히 여러 개 붙어 있음**



| 스키마 설계, 관계는 근거를 남기고 값은 근거 없이 만들지 않는다

편입과 출자관계를 n-ary 중간 노드로 개체화

- 편입과 출자는 비중, 기준일, 근거문서까지 관계 자체에 붙어야 하는 사실이기 때문에 상품에서 증권으로 화살표 하나만 굵는 방식으로는 이 값을 담을 곳이 없어 Holding과 SubsidiaryRelation을 중간 노드로 구성

ETF

```

└─ fp:hasHolding
    └─ fp:Holding
        └─ fp:holdingSecurity
        └─ fp:weight
        └─ fp:asOf
        └─ fp:sourceld
        └─ fp:supportedBy
  
```

- weight와 ownershipPct의 결측은 트리플 자체를 만들지 않음. 0으로 채우면 "지분이 0%다"라는 존재하지 않는 사실이 그래프에 생기기 때문(편입·출자 여부를 모르는 것과 편입·출자 비율이 0인 것은 전혀 다른 사실)

Company

```

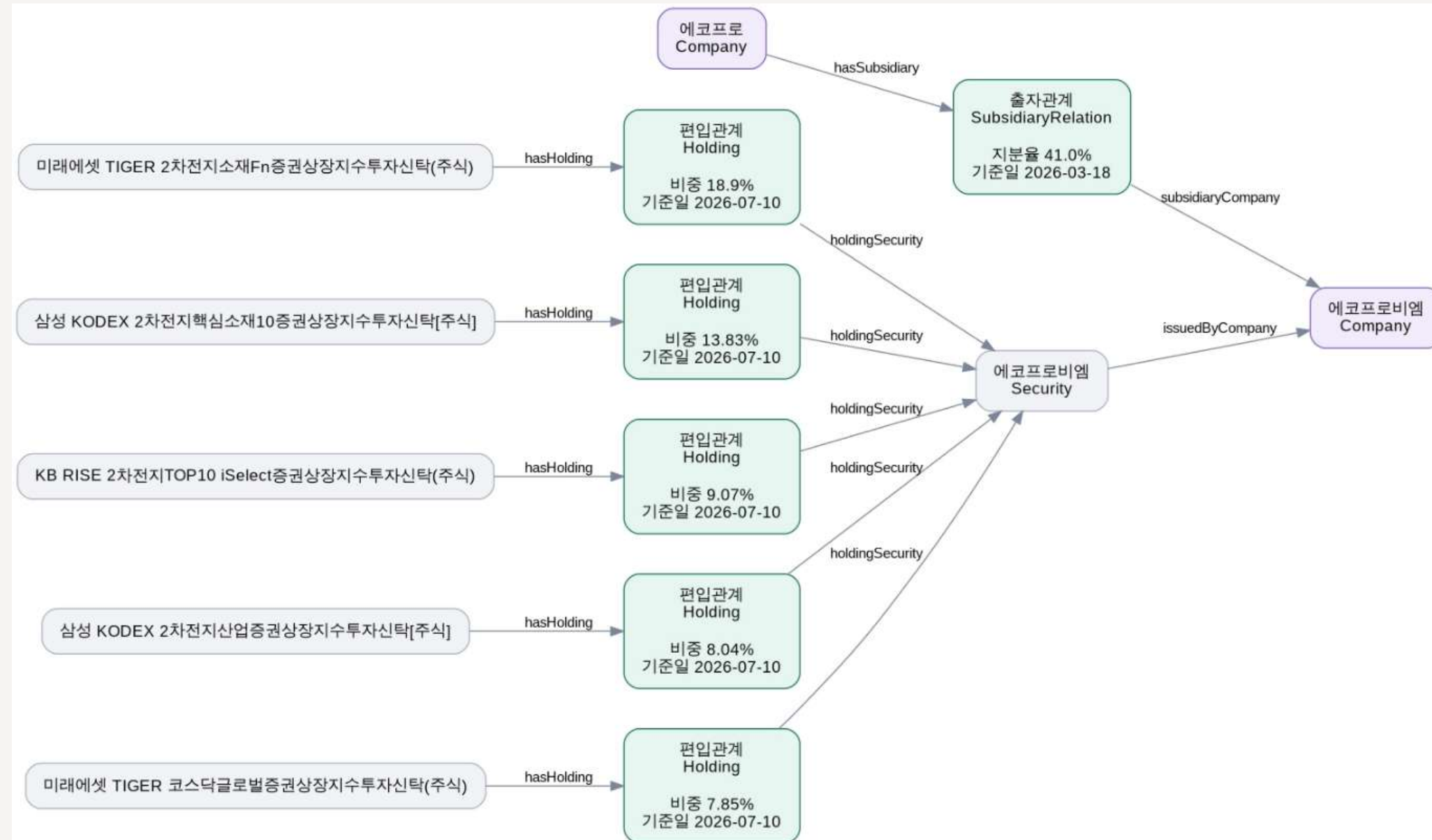
└─ fp:hasSubsidiary
    └─ fp:SubsidiaryRelation
        └─ fp:subsidiaryCompany
        └─ fp:ownershipPct
        └─ fp:asOf
        └─ fp:supportedBy
  
```

| 스키마 설계, 관계는 근거를 남기고 값은 근거 없이 만들지 않는다

예시

n-ary 구조를 사용하기 때문에 편입비중, 소유비율, 기준일, 근거문서를 관계 단위로 보존할 수 있음

- 에코프로비엠 Company : DART 법인번호 기준의 법인 그 자체 (출자·자회사 관계처럼 "회사와 회사 사이의 관계"를 표현할 때 쓰는 개체)
- 에코프로비엠 Security : 종목코드(247540)기준의 실제 거래 가능한 주식(ETF가 포트폴리오에 담는 건 회사라는 추상적 실체가 아니라 이 증권)
- 그림에서 **issuedByCompany**로 Security가 Company를 가리키는 화살표가 바로 "이 증권은 이 법인이 발행했다"는 뜻



DB 라우팅 전략

질의 유형에 따라 어떤 DB 탐색 경로를 선택했고 왜 그렇게 설계했는가

| DB 검색 흐름 결정 유형별 라우팅 예시

RDB only

Q: 신용등급 A+ 이상인 회사채를 만기일이 빠른 순으로 10개 보여줘

- 조건: **relations** 비어 있음, narrative 요구 없음, **RDB 트리거 1개 이상**
- Query Intent 결과:
 - **conditions(target)** 1건
 - **sort**={만기일, asc, 10}
 - **subtype**=[회사채]
- plan: **rdb_채권** 단일 단계
- **depends_on**=[]

GraphDB only

Q: 에코프로가 지분을 보유한 자회사를 알려줘

- 조건: **relations** 존재, RDB 트리거가 전부 없음, narrative 요구 없음
- Query Intent 결과:
 - **relations**=**[R1]**
 - **conditions** 없음
 - **sort** 없음
 - **fields** 없음
- plan: **graph_R1** 단일 단계
- **depends_on**=[]

VectorDB only

Q: 채권에 투자할 때 유의할 위험 요인을 설명해줘

- 조건: **narrative** 요구만 존재, relations 없음, RDB 트리거 없음
- Query Intent 결과:
 - **narrative_topics**=**[투자 위험]**,
 - **answer_format**=**narrative**
- plan: **vector_narrative** 단일 단계
- **depends_on**=[]

핵심 : 정렬 기준이나 출력 필드(수익률 등 정형 속성)가 하나라도 붙으면 즉시 RDB 단계 추가. 순수 관계 탐색일 때만 Graph 단독 검색

| DB 검색 흐름 결정 유형별 라우팅 예시 - 두 개 이상 연결 (직렬)

GraphDB → RDB

Q: 삼성전자를 편입한 국내ETF 중 1년 수익률 상위 5개

- plan: `graph_R1(depends_on=[])` → `rdb_국내ETF(depends_on=[graph_R1])`
- 데이터 주입: Graph가 반환한 ETF 종목코드 목록이 RDB 조회의 필터 조건으로 들어가고 그 결과에 수익률 정렬과 상위 5개 절단 적용

GraphDB 체인 → RDB

Q: 에코프로의 자회사를 편입한 국내ETF 중 수익률 상위 5개

- intent: R1(에코프로의 자회사), R2(편입 ETF, object_ref=R1)
- plan: `graph_R1` → `graph_R2(depends_on=[graph_R1])` → `rdb_국내ETF(depends_on=[graph_R2])`
- 데이터 주입: R2가 object_ref로 R1을 참조하므로 위상 정렬이 순서를 확정하고 R1은 leaf가 아니라 RDB는 R2만 기다림

RDB → GraphDB (LLM 판정)

Q: 신용등급 A+ 이상인 채권을 발행한 회사의 자회사를 편입한 ETF

- 문제: R1의 주체는 고정된 이름도 아니고 다른 relation의 결과도 아니므로 object_entity와 object_ref가 모두 비어 있어 규칙만으로는 출발점을 알 수 없음
- 처리: relation만 모아 배치로 LLM에 질의하고, needs_rdb_prestep과 rdb_domain을 받아 role=entity_lookup 단계를 앞에 삽입
- plan: `rdb_pre_R1` → `graph_R1` → `graph_R2` → `rdb_국내ETF`

Graph → RDB → Vector

Q: 삼성전자를 편입한 국내ETF TOP5와 각 상품의 투자 위험을 설명해줘

- plan: `graph_R1` → `rdb_국내ETF` → `vector_narrative`
- 데이터 주입: Vector는 항상 마지막이며, depends_on은 RDB 단계 전부, RDB가 없으면 leaf Graph 단계 전부. 앞 단계가 확정한 상품 목록으로 문서 검색 범위를 좁히는 구조

핵심 : depends_on은 앞 단계의 결과를 이 단계의 입력 필터로 주입한다는 선언. 값이 비어 있으면 즉시 실행 가능하다는 뜻

| DB 검색 흐름 결정 유형별 라우팅 예시 - 병렬

도메인 병렬

Q: 삼성전자를 보유한 국내ETF, 해외ETF, 공모펀드를 1년 수익률 TOP10

- plan:
graph_R1
↓
동시 실행
| rdb_국내ETF
| rdb_해외ETF
| rdb_펀드
- 판정 근거: 세 RDB 단계는 같은 Graph 결과에만 의존하고 서로 의존하지 않음. 도메인당 10개가 아니라 UNION ALL로 합친 뒤 재정렬하여 전체 10개를 반환해야 한다는 신호

Graph와 RDB
완전 병렬

Q: 신용등급 A+ 이상 채권 목록과, 별개로 에코프로 자회사 목록을 알려줘

- plan:
동시 출발
| graph_R1
| rdb_채권
↓
결과 합치기
- 판정 근거: relation과 RDB 조건이 서로 무관하므로 RDB가 Graph를 기다릴 이유 없음

독립 relation 병렬

Q: 우주항공 테마와 연결되고 록히드마틴을 편입한 국내ETF

- plan:
동시 출발
| graph_R1
| graph_R2
↓
rdb_국내ETF
- 판정 근거: R1과 R2가 서로를 참조하지 않아 둘 다 즉시 출발. 다만 양쪽 모두 leaf라서 후속 RDB 단계는 두 결과를 전부 기다린 뒤 수렴

핵심 : 병렬 여부는 별도로 지정하지 않고, depends_on이 비어 있는 단계는 자동으로 동시 실행되며, subject_domain 판정이 불필요한 대기를 제거

04

기대효과 및 확장성

PB·WM 상담 도우미

Hybrid DB 기반, 근거를 제시하는 금융 상담

PB/WM 상담 지원 챗봇



기대효과 - 현업 적용 지점

01

상품군을 가로지르는 확인

한 기업의 채권·ETF·펀드 노출을 한 질문으로 확인해, 상품군별 화면을 오가며 조회하던 상담 준비 과정을 단축한다.

02

드러나지 않는 중복 노출 확인

편입종목과 모자회사 관계를 따라가 서로 다른 상품에 겹쳐 있는 기업 노출을 찾아내, 고객 포트폴리오의 실질 집중도를 점검한다.

03

근거 제시와 명확한 보류

근거 문서와 기준일을 함께 제시하고 데이터에 없는 항목은 사유를 밝혀, 상담 내용을 그대로 설명 의무 이행 자료로 남길 수 있다.

확장 방향

상품정보 조회



상담 자료 작성 지원



고객별 맞춤 상담 준비 지원

05

부록

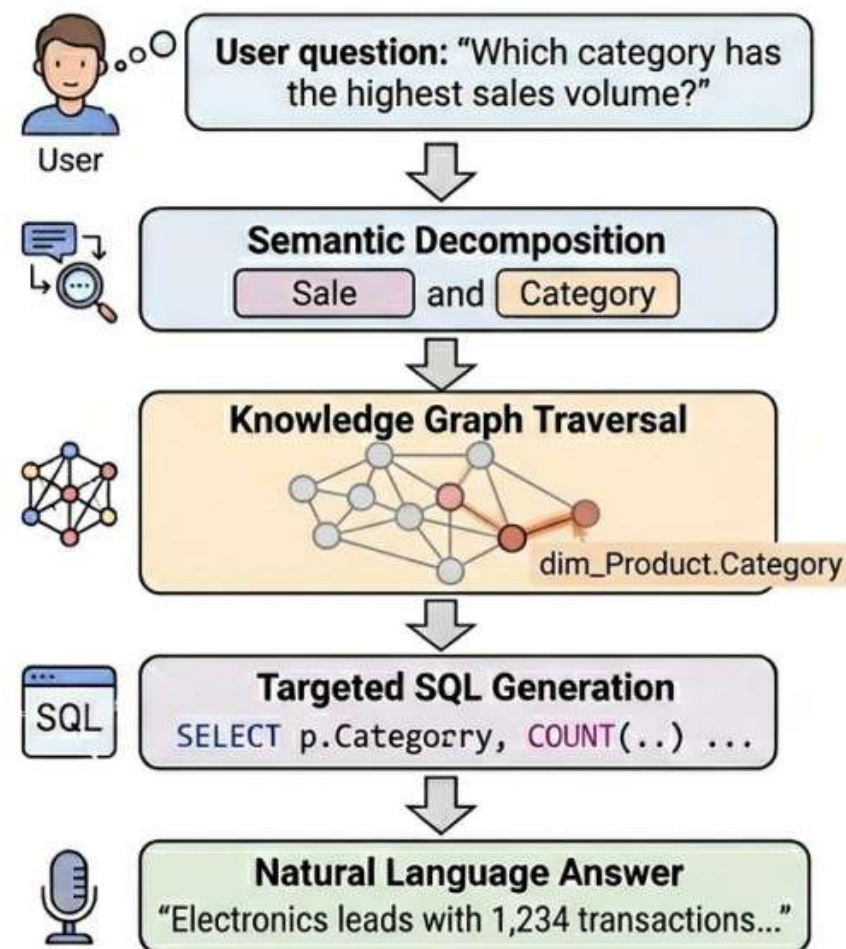
| 참고문헌 Summary

- [산업사례] Knowledge Graph-Powered NL2SQL Agent (IWConnect, 2026)
 - Ontology·Knowledge Graph 기반 Semantic Routing과 Relevant Schema 추출 구조 설계에 참고
- [논문] Knowledge Intensive Agents (AI Open, 2026)
 - 질문 유형에 따라 RDB·VectorDB·GraphDB를 선택하는 Adaptive Routing 구조 설계에 참고
- [논문] CHASE-SQL (arXiv:2410.01943, 2024)
 - Value Retrieval, SQL 후보 생성, 실행 오류 기반 Query Fixer·Retry 구조 설계에 참고
- [논문] XiYan-SQL (arXiv:2507.04701, 2025)
 - SQL 생성 전 Relevant Table·Column을 선별하는 Schema Filtering 구조 설계에 참고

| 참고문헌

• [산업사례] Knowledge Graph-Powered NL2SQL Agent (IWConnect, 2026)

Pipeline: Natural Language to Data-Driven Answer



• Semantic Routing

- 사용자 질문을 바로 SQL 생성 단계로 전달하지 않고, 질문의 의미·도메인·엔티티를 기반으로 관련 Business Concept과 DB Schema를 Knowledge Graph에서 우선 탐색하는 구조를 참고

• Ontology ↔ RDB Schema Mapping

- Ontology의 Class·Property를 실제 RDB의 Table·Column·Foreign Key와 연결하여 자연어의 금융 개념을 물리적 DB Schema로 변환하는 구조를 참고

• Relevant Schema Retrieval

- 전체 Schema를 LLM에 입력하지 않고, Knowledge Graph 탐색을 통해 질문과 관련된 Table/Column/Join 관계만 선별하여 SQL Generator에 전달하는 방식에 참고

• Graph 기반 Join Path 탐색

- LLM이 Table 간 JOIN 관계를 임의로 추론하는 대신, Knowledge Graph에 정의된 FK 및 Entity 관계를 이용해 필요한 Join Path를 사전에 탐색·제공하는 구조에 참고

| 참고문헌

- [논문] CHASE-SQL (arXiv:2410.01943, 2024)
 - Value Retrieval, SQL 후보 생성, 실행 오류 기반 Query Fixer·Retry 구조 설계에 참고
 - Query Plan CoT (실행 계획 기반 추론)
 - 데이터베이스 엔진이 내부적으로 사용하는 실행 계획(EXPLAIN 명령어 결과)의 동작 방식을 모방
 - Query Fixer (쿼리 수정 에이전트)
 - 생성된 후보 쿼리들의 문법적 오류를 잡는 일종의 Self-reflection 루프 모방(최대 수정 시도 K=3)

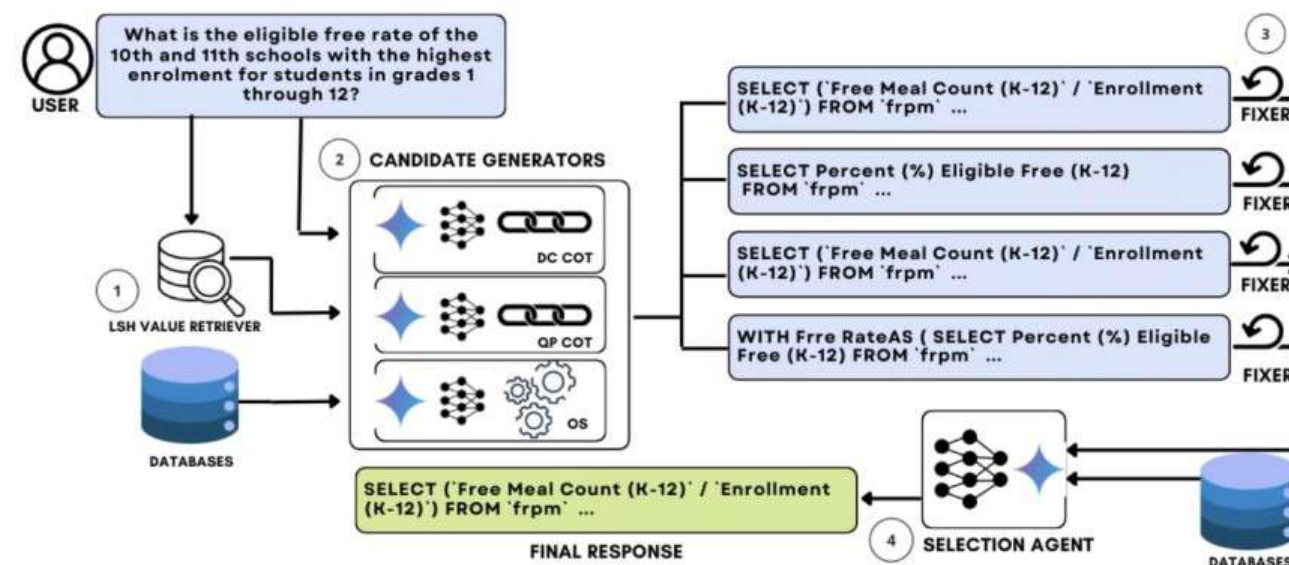


Figure 1: Overview of the proposed CHASE-SQL framework for Text-to-SQL, with value retrieval and using a selection agent for improve picking of the answers among the generated candidates along with a fixer to provide feedback for refinement of the outputs.

| 참고문헌

- [논문] XiYan-SQL (arXiv:2507.04701, 2025)
 - SQL 생성 전 Relevant Table·Column을 선별하는 Schema Filtering 구조 설계에 참고
 - Schema Linking :
 - 자연어 질문의 Entity/조건·지표를 실제 DB의 Table/Column과 연결한 뒤, 필요한 Schema만 SQL Generator에 전달하는 방식에 참고

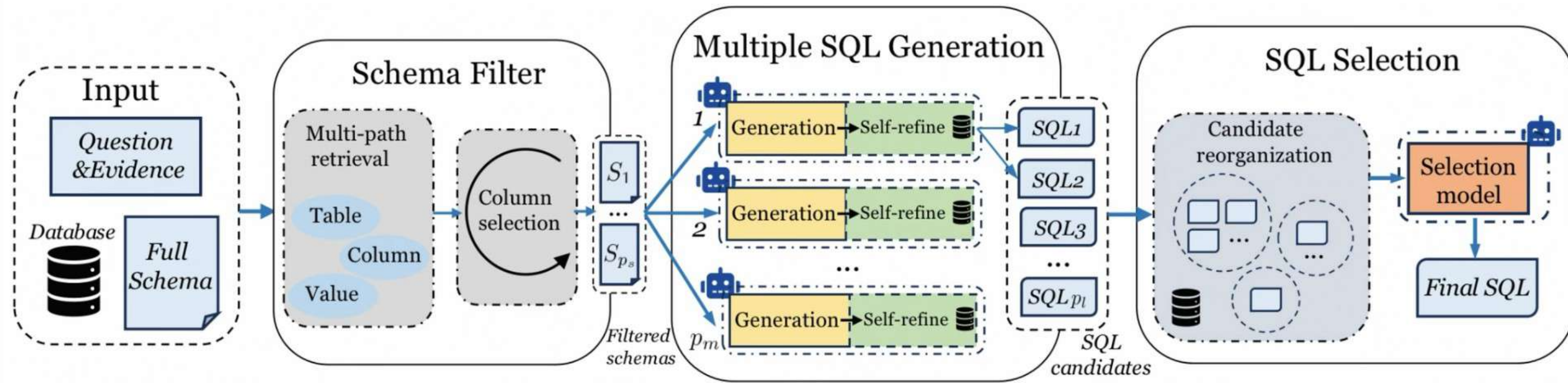


Fig. 1. Overview of the proposed XiYan-SQL framework, including three steps: Schema Filter, Multiple SQL Generation, and SQL Selection.